

Technical Task

Build a pipeline that:

1. Opens demo.mp4 twice as two independent "cameras" (cam_a, cam_b), each in its own thread via `cv2.VideoCapture(path, cv2.CAP_FFMPEG)`.
2. Decodes in real-time pacing — not faster than source FPS. Your method must also be correct on variable-FPS input (explain in REPORT, even though demo.mp4 is CFR).
3. Loops on EOF — seek to 0, continue.
4. Exposes latest-frame-wins to a single consumer thread (justify your mechanism).
5. Applies a quality gate:
 - Blur — variance of Laplacian
 - Exposure — mean luma + clipped-pixel ratio
 - Stuck frame — perceptual hash vs previous acceptedThresholds must be justified with a short experiment on demo.mp4.
6. Preprocesses accepted frames to letterboxed 640×640 NCHW float32. Two implementations required: `cv2.dnn.blobFromImage` and hand-written numpy. Both benchmarked.
7. Provides an inverse-transform function: bbox in letterbox coords → bbox in source coords. Unit test: round-trip error < 1 px.
8. Emits JSON metrics every 5 s per stream: `fps_capture`, `fps_preprocess`, `frame_age_ms_p50`, `frame_age_ms_p99`, `frames_dropped`, `recovery_count`, `quality_counters`, `rss_mb`.
9. Runs a 10-minute soak with clean SIGINT shutdown in < 2 s, no zombie threads, RSS stable.

Hard constraints

- No GPU, no cloud, no LLM.
- No `queue.Queue()` with unbounded size.
- No `time.sleep(1/fps)` as the throttle — must use frame timestamps.
- No `while True: try: ... except: pass`.
- ≤ 600 LOC total. Entry: `python run.py--input demo.mp4`.

Deliverables

- src/ — code
- REPORT.md $\leq$ 2 pages
- tests/test_letterbox_roundtrip.py — must pass
- notebooks/thresholds.ipynb or scripts/pick_thresholds.py — the experiment that justified your quality thresholds, with plots
- requirements.txt + one-line README.md with run instructions

Written answers in REPORT (2–4 sentences each)

1. How did you throttle to real-time without busy-waiting, and why does your method work on variable-FPS input?
2. Your latest-frame-wins mechanism — prove it can't deadlock when the consumer is faster, slower, or the same speed as the producer.
3. Why is cv2.dnn.blobFromImage fast? Where does your numpy version lose, and by how much?
4. What does VideoCapture.read() return on a decoding glitch — (False, None), garbage frame, or hang? How did you decide what to treat as recoverable?
5. Pick one measurement in your report you are least confident about. Explain why and how you'd strengthen it.

Things to avoid

- 10-minute soak completes in 30 seconds (no real-time pacing)
- Quality thresholds picked with no experiment
- Reporting only mean FPS, no p99
- Missing or > 1 px error on inverse-transform test
- SIGINT handler is sys.exit() leaving threads running
- Claims "stable memory" without a plot

Stretch (optional)

Replace the latest-frame-wins slot with a self-written SPSC ring buffer (no queue.Queue, no locks on the hot path). Benchmark and

show $\geq 20\%$ p99 frame-age reduction at 2 streams.

Note : Loop the file as needed to cover the 10-minute soak.

File Source : demo.mp4